



Lists & Data Frames

Follow-up Lecture — Handling Mixed Data Types and Tabular Datasets in R

Hands-on session — creation · structure · indexing · manipulation



```
> str(bio_df)
'data.frame': 6 obs. of 3 var.
> bio_df$sample_id
[1] "S1" "S2" "S3" "S4"
> patient <- list(
+   id = 101, wt = 62.4)
> patient$wt
[1] 62.4
```

Where We Left Off

Last session covered the R structures used to reach vectors, matrices and arrays

1

Vectors

One-dimensional, ordered collection of values of the same type

2

Matrices

Two-dimensional, homogeneous arrangement of elements in rows and columns

3

Arrays

N-dimensional generalization of matrices for multi-way biological data

4

Indexing

Square-bracket [] notation to access, slice and modify elements

Today we move from single-type collections to structures that hold mixed data types and tabular datasets — Lists and Data Frames

Learning Objectives

1

Create & build lists

Combine mixed data types (numeric, character, logical, vectors) into a single list

2

Index & access elements

Use `[ ]`, `[[ ]]` and `$` to retrieve and modify list components

3

Build & inspect data frames

Construct tabular datasets and examine their structure with `str()` and `summary()`

4

Apply skills hands-on

Two guided exercises using representative biological/clinical datasets



```
> px <- list(  
+   id = 1,  
+   dose_mg = c(50, 75, 100),  
+   responder = TRUE)  
  
> px$dose_mg[2]  
[1] 75  
  
> df <- data.frame(  
+   id = 1:3,  
+   wt = c(58, 62, 65))  
> str(df)  
'data.frame': 3 obs. of 2 var.
```

From Homogeneous to Heterogeneous Structures

Vectors, matrices and arrays store one data type. Lists and data frames go further — holding mixed types and tabular records, the structures used for real biological datasets.

Vector / Matrix / Array

HOLDS

Single data type only

EXAMPLE

Numeric-only gene expression matrix

List

HOLDS

Any mix of types and lengths, nested

EXAMPLE

One patient record: ID, dose vector, flag

Data Frame

HOLDS

Tabular; columns can differ in type, equal length

EXAMPLE

A sample sheet: ID, treatment, OD reading

Creating Lists

A list is an ordered collection of R objects that can differ in type and length — R's most flexible data structure

`list()`

Combine values of any type into a single list object

`names()`

Assign or view descriptive names for list elements

`length()`

Return the number of top-level elements in a list

`str()`

Display the nested structure of a list, element by element

```
> patient <- list(
+   id = 101,
+   name = "Ansh",
+   dose_mg = c(50, 75, 100),
+   responder = TRUE)

> names(patient)
[1] "id" "name" "dose_mg" ...

> length(patient)
[1] 4
```

Accessing List Elements

Three operators reach into a list — each returns a different type of result

[]

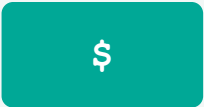
Single bracket

Returns a sub-list (keeps list structure)

[[]]

Double bracket

Returns the element itself, unwrapped from the list



Dollar sign

Returns a named element by name (partial matching allowed)

```
> patient[1]
$id
[1] 101

> patient[[1]]
[1] 101

> patient$dose_mg
[1] 50 75 100

> patient$dose_mg[2]
[1] 75
```

Modifying & Nesting Lists

Lists can be updated in place, and can even hold other lists — useful for grouped experimental records

```
patient$age <- 34
```

Add a new named element to an existing list

```
patient$id <- NULL
```

Remove an element from a list entirely

```
list(a = list(...))
```

Nest one list inside another to group related data

```
unlist()
```

Flatten a list back into a single atomic vector

```
> patient$age <- 34
> patient$id <- NULL

> cohort <- list(
+   p1 = patient,
+   p2 = list(id = 102,
+     dose_mg = c(25, 50))
+ )

> cohort$p2$dose_mg[1]
[1] 25
```

Creating Data Frames

A data frame is a table-like structure: a list of equal-length columns, each of a single data type — R's default format for biological datasets

`data.frame()`

Combine equal-length vectors into columns of a table

`read.csv()`

Import tabular biological data directly as a data frame

`nrow()` / `ncol()`

Return the number of rows or columns in a data frame

`colnames()`

View or assign column (variable) names

```
> bio_df <- data.frame(  
+   sample_id = c("S1", "S2", "S3"),  
+   treatment = c("Ctrl", "Trt", "Trt"),  
+   od600 = c(0.21, 0.58, 0.63))  
  
> nrow(bio_df)  
[1] 3  
  
> colnames(bio_df)  
[1] "sample_id" ...
```

Inspecting Data Frame Structure

`str()`**Compact display of structure**

Shows column names, types and a preview of values

`summary()`**Statistical summary per column**

Min, max, mean and quartiles for numeric columns

`head()` / `tail()`**Preview rows**

Shows the first/last n rows of the data frame

`dim()`**Overall table shape**

Returns rows and columns as `c(nrow, ncol)`

```
str(bio_df) → 'data.frame': 3 obs. of 3 variables: $ sample_id: chr $  
treatment: chr $ od600: num
```

Indexing Data Frames

Rows and columns can be selected together — base R uses [row, column] notation, alongside \$ and [[]]

```
df[2, 3]
```

Single element — row 2, column 3

```
df[, 2]
```

Entire 2nd column, all rows

```
df[1, ]
```

Entire 1st row, all columns

```
df$treatment
```

Entire column by name (returns a vector)

```
df[df$od600 > 0.5, ]
```

Logical indexing — rows meeting a condition

```
> bio_df[2, 3]
[1] 0.58

> bio_df[, 2]
[1] "Ctrl" "Trt" "Trt"

> bio_df$treatment
[1] "Ctrl" "Trt" "Trt"

> bio_df[bio_df$od600 > 0.5, ]
  sample_id treatment od600
2         S2       Trt  0.58
```

Key Takeaways

List

Ordered, mixed-type collection — `list()` · `[[ ]]` · `$` · `unlist()`

Data Frame

Tabular, equal-length columns — `data.frame()` · `str()` · `df[row, col]`

Remember: a data frame is really a list of equal-length vectors — indexing rules for lists carry over directly into data frame columns.